Chapter 10 — Responsive Design

People open the same page on a 360px-wide phone, a tablet turned sideways, a 1440px laptop and a 2560px monitor — and some of them have set a larger default font, zoomed to 200%, switched their system to dark mode or asked it to reduce motion. **Responsive design** is the practice of building *one* page that works well in all of those situations, instead of a separate "mobile site". Chapters 7–9 gave you the layout tools (flow, flexbox, grid); this chapter is about making those layouts **adapt**.

What you'll learn

#	Topic	You will be able to…
10.1	What responsive design is	name its three ingredients and explain mobile-first
10.2	The viewport meta tag and viewport units	explain why phones need it, and choose between `vh`, `svh`, `lvh`, `dvh`
10.3	Media queries	write queries with types, features, range syntax and logic, and use them from JavaScript
10.4	Mobile-first in practice	build a page that goes from 1 to 3 columns with two breakpoints
10.5	User-preference queries	support dark mode, higher contrast and reduced motion
10.6	Container queries	make a component adapt to the space *it* is given, not the screen
10.7	Fluid typography with `clamp()`	derive a fluid font-size formula and keep it accessible
10.8	Responsive images recap	pick the right image with `srcset` / `sizes` / `<picture>` and `image-set()`
10.9	Intrinsic layouts	build layouts that adapt with no breakpoints at all
10.10	Testing responsive design	test with DevTools, real devices, zoom — and automate an overflow check

Every example is a page in `code/`. The screenshots in `img/` were taken in headless Chrome by `code/make_figures.mjs`, and the `NN_*.mjs` scripts print what the browser actually computed (which queries match, resolved grid columns, font sizes). They use the helper `code/measure.mjs` — the same idea as in Chapters 7–9, extended with **viewport width**, **phone emulation** and **OS-preference emulation** (dark mode, contrast). Many screenshots are "contact sheets": the *same* page at several widths, side by side, with the width in red above each one.

10.1 What responsive design is

What it is

A **responsive** page rearranges and resizes itself to fit the **viewport** (the visible area of the browser window) and the user's settings. The term was coined by Ethan Marcotte in 2010, with three ingredients:

Ingredient	Meaning	Today's tools
Fluid grids	Widths in relative units (%, `fr`, `vw`) instead of fixed pixels	Flexbox, Grid, `min()` / `max()` / `clamp()`
Flexible media	Images and videos shrink with their container	`max-width: 100%; height: auto`, `srcset`, `aspect-ratio`
Media queries	CSS rules that apply only under conditions (e.g. "at least 640px wide")	`@media`, and now `@container`

Analogy: water takes the shape of whatever glass you pour it into. A fixed-width page is an ice cube — it fits only one glass.

Why it matters

- Most web traffic worldwide is from phones; a page that needs sideways scrolling or pinch-zoom on a phone loses readers.
- Search engines index the mobile version of a page first ("mobile-first indexing" at Google), so a broken mobile layout hurts discovery (Chapter 14).
- Accessibility law and WCAG 2.2 require content to **reflow** at 320 CSS pixels wide and survive 200% text zoom (§10.10, Chapter 3).

Mobile-first vs desktop-first

	Mobile-first	Desktop-first
Base CSS (no query)	The phone layout	The desktop layout
Queries use	`min-width` / `width >=` — *add* features as space grows	`max-width` / `width <` — *undo* features as space shrinks
Small devices download/parse	only the rules they need, plus harmless overrides	desktop rules, then overrides to undo them
Design pressure	Forces you to decide what matters most first	Tends to cram a big design into a small screen

Mobile-first is the common default today because the single-column phone layout is the simplest — normal flow already stacks things — and every larger layout is an *enhancement*. That is **progressive enhancement**: start with something that works everywhere, then add.

Responsive vs adaptive

An **adaptive** site serves (or switches between) a few fixed-width layouts; a **responsive** one is fluid *between* breakpoints. Modern practice is responsive, often with fewer breakpoints thanks to intrinsic layouts (10.9) and container queries (10.6).

Self-check

1. Name the three ingredients of responsive design. → Fluid grids, flexible media, media queries.
2. In mobile-first CSS, which kind of width query do you mostly write? → `min-width` (or `width >= …`).

10.2 The viewport meta tag and viewport units

What it is

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

This line in `<head>` (Chapter 2) tells a mobile browser: "lay the page out at the device's width in CSS pixels, at 100% zoom".

The problem it solves

When the iPhone launched (2007), almost every site was designed for ~1000px-wide desktops. So mobile browsers **pretend** to be wide: without the tag they lay the page out in a **layout viewport** of about **980px**, then zoom out so it fits the screen. Old sites looked like tiny thumbnails you could pinch-zoom — but a responsive page would never see its "phone" media queries fire, because the browser claims it is 980px wide.

A **CSS pixel** is not a hardware pixel. A phone with a **device pixel ratio** (DPR) of 3 draws each CSS pixel with 3×3 physical pixels; `width=device-width` means "as many CSS pixels as the screen is wide", e.g. 390 on a typical modern phone.

Proof

`code/01_no_viewport.html` and `code/01_viewport.html` are the same page except for the tag. `code/01_viewport.mjs` opens both on an emulated 400px phone (Puppeteer's `isMobile: true` makes Chrome honour the meta tag like a real phone):

```
01_no_viewport.html  layout 980px, visible 980px, scale 0.408, font-size 16px
01_viewport.html     layout 400px, visible 400px, scale 1, font-size 16px
```

Without the tag, the page is laid out at 980px and scaled by 400 / 980 ≈ 0.408 — so the "16px" text is really about **6.5px** on screen.

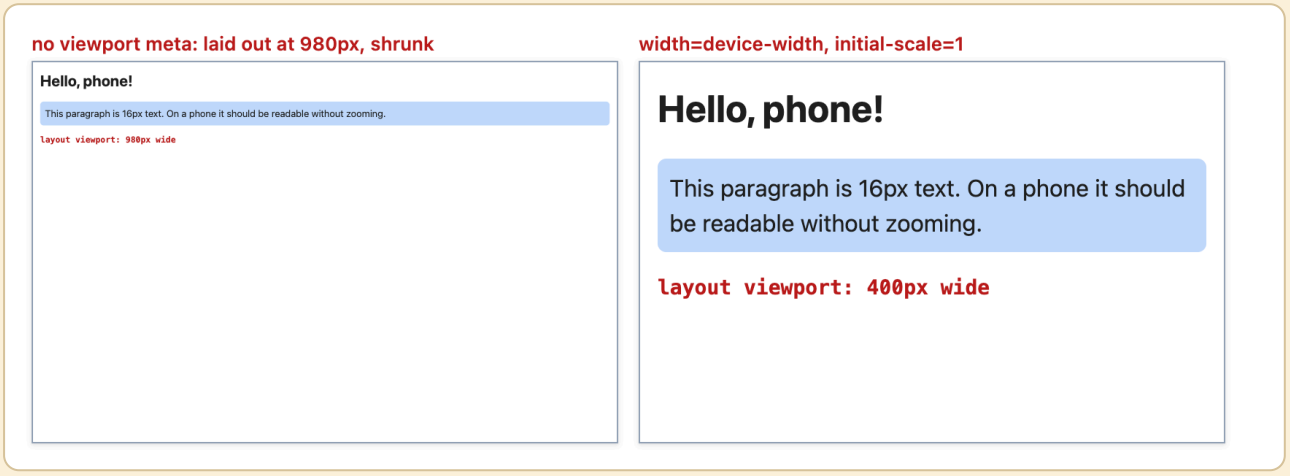


Figure 10.1 — The same page on a 400px phone, without and with the viewport meta tag.

The values

Key	Typical value	Notes
`width`	`device-width`	Layout viewport = screen width in CSS px. A number (`width=600`) is possible but rarely right.
`initial-scale`	`1`	Start at 100% zoom. Also stops iOS from changing zoom on rotation.
`maximum-scale`, `user-scalable=no`	**don't**	Blocking zoom fails WCAG 1.4.4 (Resize text). iOS Safari has ignored `user-scalable=no` since iOS 10 for this reason.
`viewport-fit`	`cover`	For notched phones; then use `env(safe-area-inset-*)` padding.

Viewport units

`vw` and `vh` are 1% of the viewport's width and height (Chapter 6 introduced the units). On phones the viewport height **changes** as the address bar slides in and out, so CSS Values 4 added three variants (Baseline since December 2022):

Unit	1% of…	Behaviour on a phone
`svh` / `svw` (small)	the viewport with browser toolbars **shown**	never too tall; may leave a gap when toolbars hide
`lvh` / `lvw` (large)	the viewport with toolbars **hidden**	may be hidden behind toolbars at first
`dvh` / `dvw` (dynamic)	the viewport **right now**	follows the toolbars; can cause relayout while scrolling
`vh` / `vw` (classic)	in practice the **large** viewport on mobile	why `height: 100vh` "overflows" on phones

Rules of thumb: `min-height: 100svh` or `100dvh` for full-screen heroes and the sticky footer of §8.7; `vw` for fluid type (10.7), but never alone — it ignores zoom settings. On desktop all four heights are equal. Also available: `vi`/`vb` (inline/block direction) and `vmin`/`vmax`.

Common mistakes

- Forgetting the tag: every phone breakpoint silently fails.
- Disabling zoom "to feel like an app" — an accessibility failure.
- `width: 100vw` on a page with a vertical scrollbar: on desktop `100vw` *includes* the scrollbar, so the element is ~15px too wide and causes sideways scrolling. Use `width: 100%`.

Self-check

1. Without the tag, how wide does a phone lay the page out? → About 980 CSS px (then zooms out).
2. Which unit should a full-height mobile hero use to avoid being covered by the address bar? → `svh` (always fits) or `dvh` (tracks the toolbar).
3. Why avoid `100vw` for full-width elements? → It includes the scrollbar width on desktop and causes horizontal overflow.

10.3 Media queries

What it is

A **media query** is a condition about the device or environment. Rules inside `@media (condition) { … }` apply only while the condition is true, and the browser re-evaluates it live (e.g. when you rotate a phone or resize a window).

```
@media screen and (width >= 40em) { .cards { grid-template-columns: repeat(2, 1fr); } }
```

Media queries don't add specificity; inside them normal cascade rules apply (Chapter 6) — which is why mobile-first queries are written **after** the base rules they override.

Where you can use them

Place	Example
CSS at-rule	`@media (width >= 40em) { … }`
`<link>`	`<link rel="stylesheet" href="print.css" media="print">` — still downloaded, but not render-blocking when it doesn't match
`@import`	`@import url(wide.css) (width >= 64em);`
`<source>` in `<picture>`	`<source media="(width >= 800px)" srcset="wide.jpg">` (Chapter 4)
JavaScript	`matchMedia('(width >= 64em)')`

Anatomy

1. **Media type** (optional): `all` (default), `screen`, `print`. Other types (`tv`, `handheld`…) are deprecated.
2. **Media features** in parentheses: `(width >= 40em)`, `(orientation: landscape)`, `(hover: hover)`.
3. **Logic**: `and`; a comma means **or**; `not` negates a whole query (or a condition in Level 4 syntax: `not (hover: hover)`); `only` is a legacy guard for very old browsers — harmless, no longer needed.

Media features you'll actually use

Feature	Values / example	Use
`width` (+ `min-` / `max-`)	`(width >= 40em)`	Layout breakpoints (the viewport width)
`height`	`(height < 500px)`	Short landscape phones: shrink sticky headers
`orientation`	`portrait` | `landscape`	Rarely needed — width usually says more
`aspect-ratio`	`(aspect-ratio >= 16/9)`	Full-screen media layouts
`hover` / `any-hover`	`hover` | `none`	Only show hover-reveal UI when the *primary* (or *any*) pointer can hover
`pointer` / `any-pointer`	`fine` | `coarse` | `none`	Larger tap targets for touch (`coarse`)
`resolution`	`(resolution >= 2x)`	High-DPR assets (prefer `srcset` / `image-set()`)
`prefers-color-scheme`	`light` | `dark`	Dark mode (10.5)
`prefers-reduced-motion`	`reduce` | `no-preference`	Tone down animation (10.5, Chapter 12)
`prefers-contrast`	`more` | `less` | `custom` | `no-preference`	Stronger borders/colours
`forced-colors`	`active` | `none`	Windows High Contrast / contrast themes
`print` (type)	`@media print`	Hide nav, show link URLs, black on white

Range syntax (Media Queries Level 4)

The old style uses `min-`/`max-` prefixes; the new **range syntax** uses comparison operators. It is Baseline since March 2023 and *widely available* since September 2025 (Chrome 104, Firefox 102, Safari 16.4).

Old	Range syntax
`(min-width: 640px)`	`(width >= 640px)`
`(max-width: 639.98px)`	`(width < 640px)`
`(min-width: 640px) and (max-width: 1023.98px)`	`(640px <= width < 1024px)`

Why the `.98`? With `max-width: 639px` and `min-width: 640px`, a window 639.5px wide (possible with zoom or fractional DPR) matches **neither**. Range syntax's `<` removes that gap.

`em` or `px` for breakpoints?

In media queries, `em` and `rem` are always relative to the **browser's default font size** (normally 16px), *not* your `html { font-size }`. So `40em` = 640px at defaults, but if a user sets 20px as their default, `40em` becomes 800px and your layout switches to one column earlier — exactly what a user with large text needs. Both units are fine; `em` ties breakpoints to text size.

Choosing breakpoints

- Let the **content** decide: widen the window until the layout looks wrong, and add a breakpoint *there*. Device-based lists go stale.

- For reference, popular frameworks use: Tailwind `sm 640 · md 768 · lg 1024 · xl 1280 · 2xl 1536`, Bootstrap 5 `576 · 768 · 992 · 1200 · 1400`.
- Fewer is better: two or three layout breakpoints, plus intrinsic techniques (10.9) and container queries (10.6) for components.

Media queries in JavaScript

`window.matchMedia(query)` returns a `MediaQueryList` with `.matches` and a `change` event — useful for behaviour that CSS can't express (swap a component, pause a carousel). Don't listen to `resize` and compare `innerWidth` yourself; `change` fires only when the answer flips.

```
const desktop = matchMedia('(width >= 64em)');
desktop.addEventListener('change', e => console.log(e.matches ? 'desktop' : 'not desktop'));
```

The last line of 10.4's output shows this firing: resizing 400 → 900 → 1100 → 1300 → 700 fired exactly **two** events. (`addListener()` is the deprecated older form.)

Common mistakes

- Desktop-first `max-width` queries mixed with mobile-first `min-width` ones — overlapping rules and gaps. Pick one direction.
- Putting a media query **before** the base rule it should override: equal specificity, so the later base rule wins.
- Using `(hover: hover)` alone to guess "desktop": laptops with touchscreens and phones with mice exist. Design for both.
- Breakpoints for specific devices ("iPhone 15") instead of the content.

Self-check

1. Write "between 600px and 900px, excluding 900" in range syntax. → `@media (600px <= width < 900px)`.
2. Does `em` in a media query follow `html { font-size: 62.5% }`? → No — it uses the browser's initial font size.
3. How does JS find out when a breakpoint is crossed? → `matchMedia(q).addEventListener('change', …)`.

10.4 Mobile-first in practice

The page

`code/02_mobile_first.html` is a small travel page. The base styles (no query) are the phone layout; two `min-width`-style queries add columns. The red badge at the top shows which rule set is active (it is a `::after` whose `content` each query overrides).

```
/* Base = phone: single column; normal flow + grid gap does the stacking */
.layout { display: grid; gap: 14px; padding: 14px; }
.cards  { display: grid; gap: 10px; }

@media (width >= 40em) {                      /* 640px: tablets, small laptops */
  header { display: flex; justify-content: space-between; align-items: center; }
  .cards { grid-template-columns: repeat(2, 1fr); }
}
@media (width >= 64em) {                      /* 1024px: sidebar appears */
  .layout { grid-template-columns: 1fr 240px; max-width: 1200px; margin-inline: auto; }
  .cards  { grid-template-columns: repeat(3, 1fr); }
}
```

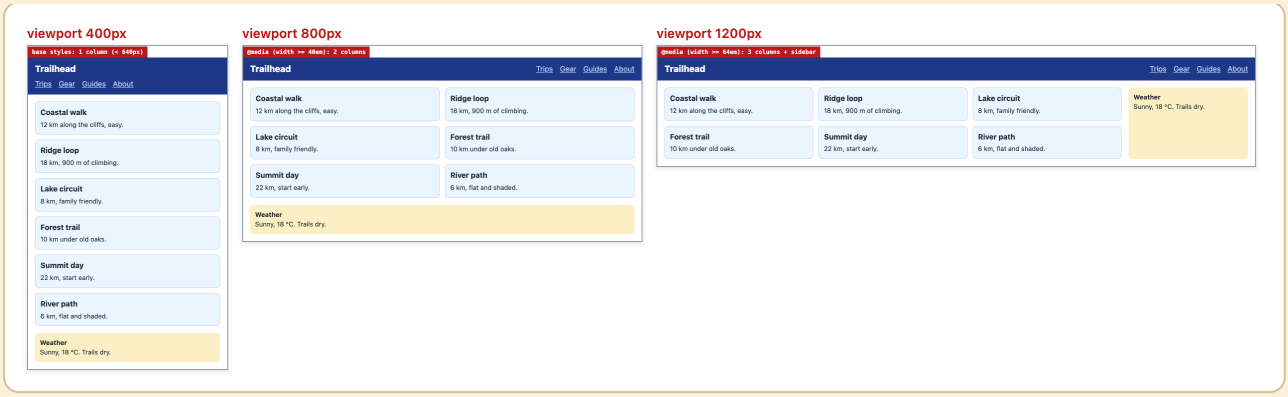


Figure 10.2 — One page, three viewport widths. Nothing in the HTML changes; only which rules apply.

Output of `code/02_media_queries.mjs` — which queries match and what the browser resolved the card grid to:

```
 400px  (width >= 40em) no   (width >= 64em) no   cards: 372px
 800px  (width >= 40em) yes  (width >= 64em) no   cards: 381px 381px
1200px  (width >= 40em) yes  (width >= 64em) yes  cards: 299.328px 299.328px 299.344px
matchMedia events: [ 'change -> desktop at 1100px', 'change -> not desktop at 700px' ]
```

Check by hand at 1200px: `.layout` is `max-width: 1200px` minus 2 × 14px padding = 1172px; minus the 240px sidebar and a 14px gap = 918px for `.cards`; minus two 10px gaps = 898px; ÷ 3 = **299.33px** per card column. ✓

Why it works

- **Normal flow** is already mobile-first: block elements stack. The base CSS mostly styles, it barely lays out.
- Each query **adds** a layout; nothing has to be "undone".
- The **DOM order** (header, cards, aside) is the reading order at every width, so keyboard and screen-reader users get the same sequence (Chapter 3). Reordering visually with `order` or grid placement at a breakpoint breaks that — avoid it for meaningful content.

Responsive navigation (common pattern)

At phone width a long nav usually becomes a **menu button** that shows/hides the list. Accessible recipe: a real `<button aria-expanded="false" aria-controls="menu">` that toggles the list (a few lines of JS), or the Popover API (`popover` attribute, Baseline since January 2025) for a no-JS disclosure. The flex-wrap nav in the demo is the simplest option when there are only a few links.

Responsive tables

Wide data tables can't reflow into one column without losing their meaning. Wrap them in a scroll container: `<div class="table-wrap" tabindex="0" role="region" aria-label="Prices">` with `.table-wrap { overflow-x: auto; }` — WCAG allows two-dimensional scrolling for data tables (Chapter 5).

Self-check

1. At 800px, which of the two queries match? → Only `(width >= 40em)`.
2. Why write the queries after the base rules? → Same specificity, so the later rule wins the cascade.

10.5 User-preference media queries

What it is

Operating systems let people choose a dark theme, reduce motion, or increase contrast. The **user-preference media features** (Media Queries Level 5) expose those choices to CSS. All of the following are Baseline:

Feature	Baseline	Typical use
`prefers-color-scheme: dark`	widely available (2020)	Dark theme
`prefers-reduced-motion: reduce`	widely available (2020)	Remove or soften animation — full treatment in Chapter 12
`prefers-contrast: more`	widely available (2022)	Solid borders, stronger colours
`forced-colors: active`	widely available (2022)	Windows contrast themes replace your colours; restore borders on custom controls, use system colours like `CanvasText`
`prefers-reduced-transparency`	**not** Baseline (Chromium only)	Remove frosted-glass effects
`prefers-reduced-data`	not supported anywhere yet	—

Dark mode — the essentials

1. **Declare support**: `<meta name="color-scheme" content="light dark">` in the head and/or `:root { color-scheme: light dark; }`. The browser then themes its **own** UI — default page background, form controls, scrollbars — and avoids a white flash in dark mode. (`color-scheme`: Baseline 2022.)
2. **Colour tokens** as custom properties (§6.9), light values first.
3. **Override the tokens** inside `@media (prefers-color-scheme: dark)`. Components only ever use the tokens.
4. For one-off values, `light-dark(lightValue, darkValue)` picks based on the element's **used colour scheme** — no media query. It needs `color-scheme: light dark` to be set, and it is Baseline since May 2024 (newly available).

```
:root { color-scheme: light dark;
        --bg: #ffffff; --text: #1f2937; --card: #eff6ff; --accent: #1d4ed8; --border: #bfdbfe; }
@media (prefers-color-scheme: dark) {
  :root { --bg: #0f172a; --text: #e2e8f0; --card: #1e293b; --accent: #93c5fd; --border: #334155; }
}
@media (prefers-contrast: more) { :root { --border: currentColor; } }

body  { background: var(--bg); color: var(--text); }
.note { color: light-dark(#92400e, #fcd34d); }
```

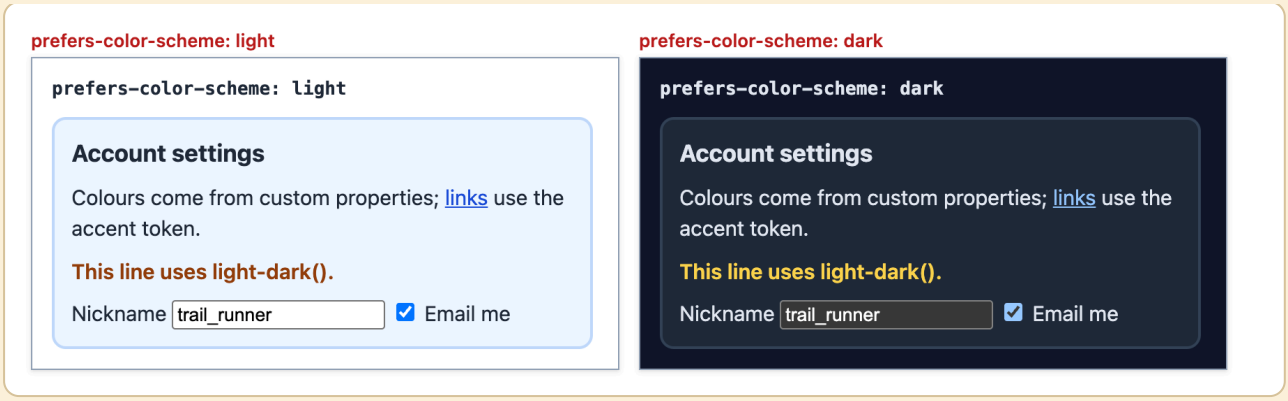


Figure 10.3 — `code/03_preferences.html` under emulated `prefers-color-scheme: light` and `dark`. Notice the text input and checkbox: nobody styled them — `color-scheme` made the browser draw its dark versions. The dark screenshot alone can be reproduced with `node tools_web_shot.mjs chapters/10-responsive-design/code/03_preferences.html out.png 420 --dark`.

`code/03_preferences.mjs` reads the computed colours under three emulated settings:

```
light (default)        bg rgb(255, 255, 255)  text rgb(31, 41, 55)  card border rgb(191, 219, 254)
note rgb(146, 64, 14)  <input> bg rgb(255, 255, 255)
dark                   bg rgb(15, 23, 42)  text rgb(226, 232, 240)  card border rgb(51, 65, 85)  note
rgb(252, 211, 77)  <input> bg rgb(59, 59, 59)
light + more contrast  bg rgb(255, 255, 255)  text rgb(31, 41, 55)  card border rgb(31, 41, 55)  note
rgb(146, 64, 14)  <input> bg rgb(255, 255, 255)
```

With more contrast, the card border became the text colour (`currentColor`). The input background `rgb(59, 59, 59)` in dark mode is Chrome's built-in dark control colour.

A practical note from making these figures: headless Chrome follows the **computer's** theme when nothing is emulated. On a Mac in dark mode, a page with `color-scheme: light dark` screenshots dark unless you emulate `light` explicitly — `measure.mjs` does exactly that.

Letting users override

Many sites add a theme toggle that sets `data-theme="dark"` on `<html>` and stores the choice. Then tokens are defined for `[data-theme="dark"]` too, and the media query applies only when no choice was made (`:root:not([data-theme="light"])`). That architecture — plus `color-scheme` per theme — is covered in Chapter 13.

Designing a dark theme well

- Don't invert: use dark greys/navies (pure `#000` with pure white text causes halation for many readers), lighten accent colours, and re-check contrast ≥ 4.5:1 (Chapter 11).
- Shadows barely show on dark backgrounds — express elevation with slightly lighter surfaces.
- Images: dim very bright photos slightly (`filter: brightness(.9)`) or supply dark versions with `<picture><source media="(prefers-color-scheme: dark)" …>`.

Reduced motion in one rule

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation-duration: 0.01ms !important; animation-iteration-count: 1
!important;
                           transition-duration: 0.01ms !important; scroll-behavior: auto !important; }
}
```

This blunt reset is common; §12.9 explains a gentler "motion only when allowed" pattern and what WCAG requires.

Self-check

1. What does `color-scheme: light dark` change even if you write no other dark CSS? → The browser's defaults: canvas background, text colour, form controls, scrollbars.
2. When does `light-dark()` pick its second value? → When the element's used colour scheme is dark (requires `color-scheme` to allow dark).
3. What should a site do when `forced-colors: active`? → Let the system colours win; make sure custom controls still have visible borders/focus indicators (e.g. with system colours like `ButtonText`), and don't rely on background images for meaning.

10.6 Container queries

What it is

A **container query** (`@container`) styles an element based on the size of an **ancestor container** instead of the viewport. It is Baseline since February 2023 and **widely available** since August 2025 (Chrome 105, Firefox 110, Safari 16).

The problem it solves

Components are reused in different places: a product card sits in a wide main column, a narrow sidebar and a two-up grid — all on the same screen. A media query only knows the *viewport* width, so it can't tell those three cards apart. You used to add modifier classes (`.card--compact`) by hand. With container queries the card adapts to the space **it is given**, wherever it is placed.

Analogy: media queries ask "how big is the room?"; container queries ask "how big is the box I'm sitting in?"

How it works

1. Mark an ancestor as a **size container**: `container-type: inline-size` (width only — the usual choice) or `size` (both axes, but then the container can't get its height from its content, so you must give it one). The shorthand `container: name / type` also sets an optional **name**.
2. Write rules inside `@container [name] (condition) { … }`. They apply to **descendants** of the matching container.
3. The browser finds, for each element, the **nearest ancestor** container (with that name, if given) and evaluates the condition against **its** content-box size.
4. A container can't style *itself* with its own query — that would be circular. This is why the demo puts the container on a wrapper (`.card-slot`) and styles the `.product` inside.

```
.card-slot { container: card / inline-size; }       /* name "card", measured inline */

.product    { display: grid; gap: 8px; }             /* base: picture above text */
.product h3 { font-size: clamp(15px, 3cqi + 8px, 24px); }

@container card (width >= 400px) { .product { grid-template-columns: 160px 1fr; } }
@container card (width >= 600px) { .product { grid-template-columns: 240px 1fr; padding: 16px; } }
```

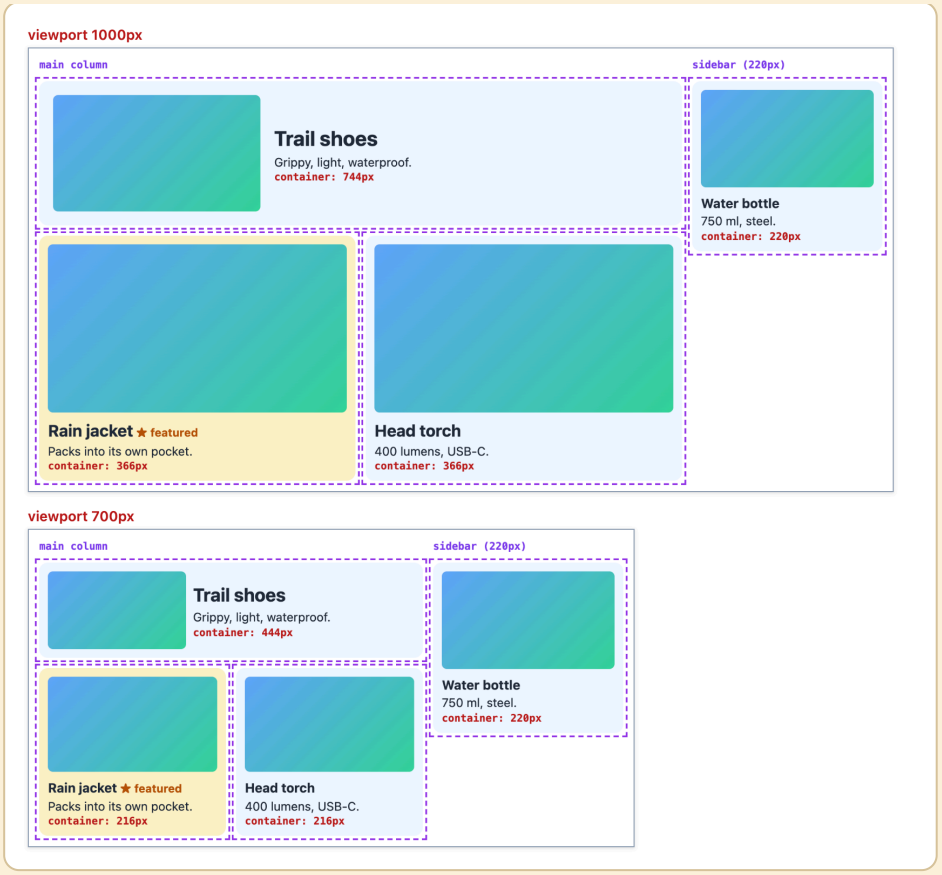


Figure 10.4 — `code/04_container_queries.html`. Dashed purple = the containers; each card prints its container width in red. Same component, same viewport, three different layouts.

Output of `code/04_container_queries.mjs`:

```
viewport 1000px
  Trail shoes   container 744px  columns 240px 456px    h3 24px      ::after none
  Rain jacket   container 366px  columns 346px          h3 18.98px   ::after " ★ featured"
  Head torch    container 366px  columns 346px          h3 18.98px   ::after none
  Water bottle  container 220px  columns 200px          h3 15px      ::after none
viewport 700px
  Trail shoes   container 444px  columns 160px 256px    h3 21.32px   ::after none
  Rain jacket   container 216px  columns 196px          h3 15px      ::after " ★ featured"
  Head torch    container 216px  columns 196px          h3 15px      ::after none
  Water bottle  container 220px  columns 200px          h3 15px      ::after none
```

The sidebar card (220px container) looks identical at both viewport widths, while the "Trail shoes" card switches from the ≥600px layout to the ≥400px layout. A media query could not do this.

Container query units

Inside a container you can size things relative to it:

Unit	1% of the query container's…
`cqw` / `cqh`	width / height
`cqi` / `cqb`	inline size / block size (writing-mode aware — prefer these)
`cqmin` / `cqmax`	smaller / larger of the two

If no suitable container exists, they fall back to the **small viewport** units (`svw`/`svh`…). Check the heading sizes: `clamp(15px, 3cqi + 8px, 24px)` in a 366px container = 3 × 3.66 + 8 = **18.98px** ✓; in a 220px container = 14.6px → clamped up to **15px** ✓; in 744px = 30.3px → capped at **24px** ✓.

Style queries

`@container style(--featured: yes)` tests a **custom property's value** on the container instead of its size. Every element is a style container by default, so no `container-type` is needed. In the demo, `.featured { --featured: yes; }` turns the "Rain jacket" card yellow with a star — a design-token switch without a modifier class on the component. Status: **Baseline since May 2026** (newly available: Chrome 111, Safari 18, Firefox 151), and only for **custom properties**; querying ordinary properties (e.g. `style(display: flex)`) is not supported. Treat it as an enhancement for a while.

Other container queries (not Baseline)

`scroll-state()` queries — "is this sticky header stuck?", "is this carousel item snapped?" — are Chromium-only (Chrome 133+). Leave them out of anything that must work everywhere.

Container queries vs media queries

Use a media query for…	Use a container query for…
Page-level layout (how many columns the page has)	Components that appear in different-sized slots
User preferences (`prefers-*`), print, pointer type	Adapting a card/widget's internal layout
Anything about the device	Sizing text/spacing relative to the component (`cqi`)

They work together: media queries shape the page, container queries let each piece adapt to the space the page gave it.

Common mistakes

- Putting `container-type` on the element you want to restyle — it can only restyle its **descendants**.
- `container-type: size` on an element without an explicit height: it collapses to 0 height, because size containment ignores its content.
- Forgetting that `inline-size` containment means the container's **width** can't depend on its children — a `container-type: inline-size` element inside a shrink-to-fit parent (a flex item with `flex: none`, an inline-block, a float) may collapse. Give it a width from outside (grid track, block flow, `flex: 1`).
- Naming: an unnamed `@container (width > 400px)` matches the **nearest** size container of any name, which might not be the one you meant. Name containers in design systems.

Fallback for older browsers

```
@supports not (container-type: inline-size) { .product { /* a sensible single layout */ } }
```

`@supports (feature: value)` is a **feature query**: its rules apply only if the browser understands that declaration. Container queries are widely available now, so this is only needed for old devices.

In the interview

30-second answer: "Media queries respond to the viewport; container queries respond to the size of an ancestor that I've declared a container with `container-type: inline-size`. That makes components portable: the same card can be stacked in a sidebar and side-by-side in the main column on the same screen. The container can't query itself, so I put it on a wrapper, and I use `cqi` units for sizes relative to it. Size queries are Baseline since 2023; style queries on custom properties became Baseline in 2026."

Follow-ups:

- *Why can't the container use `container-type: size` by default?* → Size containment means its size can't depend on its content; for height that would usually collapse it to 0. `inline-size` only fixes the width dependency.
- *What happens to `cqi` if there's no container?* → It falls back to small-viewport units (`svi`).
- *Would you drop media queries?* → No: page layout and user preferences still belong to media queries.

Self-check

1. Where does `container-type` go: on the card or its wrapper? → On the wrapper (an ancestor); the card's styles come from queries about the wrapper.
2. A container is 500px. What is `10cqi`? → 50px.
3. What does `@container style(--theme: dark)` test? → Whether the nearest container's computed `--theme` custom property equals `dark`.

10.7 Fluid typography with `clamp()`

What it is

Fluid typography makes font sizes (and often spacing) change **smoothly** with the viewport between a minimum and a maximum, instead of jumping at breakpoints. The tool is `clamp(MIN, PREFERRED, MAX)` — Baseline since 2020 (now widely available), along with `min()` and `max()`:

- `clamp(MIN, VAL, MAX)` = `max(MIN, min(VAL, MAX))`: use `VAL`, but never below `MIN` or above `MAX`.

How to derive the formula

Goal: body text **16px at a 320px viewport** growing to **24px at a 1200px viewport**.

1. **Slope** = change in size ÷ change in viewport = (24 − 16) / (1200 − 320) = 8 / 880 = 0.00909. Since `1vw` = 1% of the viewport, that is **0.909vw**.
2. **Intercept** (the size at a 0px viewport) = 16 − 0.00909 × 320 = 13.09px = **0.818rem**.
3. Preferred value = `0.818rem + 0.909vw`, clamped to `1rem … 1.5rem`:

```
:root {
  --step-0: clamp(1rem, 0.818rem + 0.909vw, 1.5rem);      /* 16px → 24px  */
  --step-3: clamp(1.75rem, 1.296rem + 2.273vw, 3rem);     /* 28px → 48px  */
  --space:  clamp(0.75rem, 0.341rem + 2.045vw, 1.875rem); /* 12px → 30px  */
}
p  { font-size: var(--step-0); }
h1 { font-size: var(--step-3); }
```

`code/05_fluid_type.mjs` compares Chrome's computed sizes with the straight-line formula:

```
slope = 0.909vw, intercept = 13.091px = 0.818rem
  280px  p 16.00px (by hand 16.00)   h1 28.00px
  320px  p 16.00px (by hand 16.00)   h1 28.01px
  600px  p 18.54px (by hand 18.55)   h1 34.37px
  900px  p 21.27px (by hand 21.27)   h1 41.19px
 1200px  p 24.00px (by hand 24.00)   h1 48.00px
 1600px  p 24.00px (by hand 24.00)   h1 48.00px
default font 20px, viewport 1000px, rem + vw (ours)  -> 25.45px
default font 20px, viewport 1000px, vw only (bad)    -> 16px
```

The 0.01px differences come from rounding the CSS coefficients to three decimals.

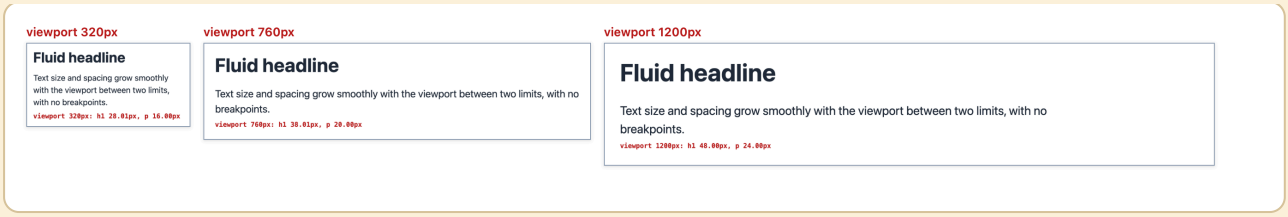


Figure 10.5 — `code/05_fluid_type.html` at three widths. Each page prints its own computed sizes.

Why the `rem` part is essential (accessibility)

The last two output lines emulate a user who changed the browser's **default font size** to 20px. The `rem + vw` formula grows to 25.45px (0.818 × 20 + 9.09). A pure `vw` size stays at **16px** — it ignores the user's setting, and it also doesn't grow with **page zoom** in the way users expect. WCAG 2.2 SC 1.4.4 (Resize text) requires text to scale to 200%; keep a `rem` term in the preferred value, keep `MIN` and `MAX` in `rem`, and keep the ratio MAX/MIN at or below about 2.5 so that zooming still enlarges text meaningfully.

Type scales

Designers usually pick a **modular scale** — each step = previous × ratio (e.g. 1.2 "minor third" on small screens, 1.333 "perfect fourth" on large). Fluid type interpolates **each step** between its small-screen and large-screen size, which is what `--step-0 … --step-5` token sets from generators such as Utopia do. The same interpolation works for spacing (`--space`), making padding and gaps breathe with the screen.

Common mistakes

- `font-size: 4vw` alone — unreadable on phones, huge on monitors, ignores user settings.
- Mixing units in the wrong places: `clamp(16px, …, 24px)` ignores the user's default for the limits; prefer `rem`.
- Forgetting `line-height`: large headings need a smaller unitless line-height (1.1–1.2), body text ~1.5 (Chapter 11).

Self-check

1. Derive the preferred value for 18px at 400px → 22px at 1400px. → Slope 4/1000 = 0.4vw; intercept 18 − 1.6 = 16.4px = 1.025rem → `clamp(1.125rem, 1.025rem + 0.4vw, 1.375rem)`.
2. Why not `clamp(1rem, 2vw, 1.5rem)`? → It works, but the middle term has no `rem` part, so between the limits it ignores the user's font-size setting.

10.8 Responsive images recap (and `image-set()`)

Chapter 4 §4.3 covers responsive images in depth. The short version:

Need	Tool
Image never overflows its container	`img { max-width: 100%; height: auto; }` + `width`/`height` attributes (reserves space → no layout shift)
Same picture, different file sizes; browser picks by layout width and DPR	`<img srcset="a-400.jpg 400w, a-800.jpg 800w" sizes="(width >= 64em) 33vw, 100vw">`
Same size on screen, sharper on high-DPR screens	`srcset="logo.png 1x, logo@2x.png 2x"`
Different crop at different widths (**art direction**)	`<picture><source media="(width >= 800px)" srcset="wide.jpg"><img src="square.jpg" alt="…"></picture>`
Newer format with fallback	`<picture><source type="image/avif" srcset="a.avif"><img src="a.jpg" alt="…"></picture>`
Keep a box's shape before content loads	`aspect-ratio: 16 / 9`

`image-set()` for CSS background images

`srcset` only works on `<img>`. For **background images**, CSS has `image-set()` — Baseline since September 2023, widely available since March 2026 (older Safari needed `-webkit-image-set()`):

```
.hero {
  background-image: url("media/hills-400.jpg");                /* fallback */
  background-image: image-set("media/hills-400.jpg" 1x,
                              "media/hills-800.jpg" 2x);
  background-size: cover;
}
```

Candidates can also be chosen by format, e.g. `image-set("a.avif" type("image/avif"), "a.jpg" type("image/jpeg"))`. `code/06_image_set.mjs` loads `code/06_image_set.html` at three pixel ratios and logs the network:

```
devicePixelRatio 1: downloaded hills-400.jpg
devicePixelRatio 2: downloaded hills-800.jpg
devicePixelRatio 3: downloaded hills-800.jpg
```

Only **one** file is downloaded per screen — the whole point. A background image is decoration: if the picture carries meaning, use an `<img>` with `alt` (the demo adds `role="img"` and `aria-label` only because it is a demo of a decorative technique).

10.9 Intrinsic layouts — responsive without breakpoints

What it is

Intrinsic design (a term from Jen Simmons) means letting the layout algorithms respond to the available space themselves, using `min()`, `max()`, `clamp()`, `flex-wrap`, `auto-fit` and `minmax()`. Fewer breakpoints means fewer "in-between" widths where things look odd, and components that work in any container.

Three patterns

`code/07_intrinsic.html` — no `@media` anywhere:

```css
/* 1. Readable centred column: at most 60ch, and never touching the screen edges */
.wrapper { width: min(100% - 2rem, 60ch); margin-inline: auto; }

/* 2. Sidebar that wraps when the content would get narrower than 60% */
.with-sidebar          { display: flex; flex-wrap: wrap; gap: 10px; }
.with-sidebar > .side  { flex: 1 1 150px; }                    /* wants ~150px */
.with-sidebar > .content { flex: 999 1 0; min-inline-size: 60%; } /* takes the rest */

/* 3. Card grid: as many ≥140px columns as fit (Chapter 9 §9.3) */
.grid { display: grid; gap: 10px; grid-template-columns: repeat(auto-fit, minmax(min(140px, 100%),
1fr)); }
```

How pattern 2 works: the content's huge `flex-grow` (999) means it takes almost all free space, so the sidebar stays near its 150px basis. When the line can't hold `150px + gap + 60%`, flex-wrap moves the sidebar to its own line, where it grows to full width. That is a "breakpoint" chosen by the content, not the viewport — and it works inside any container.

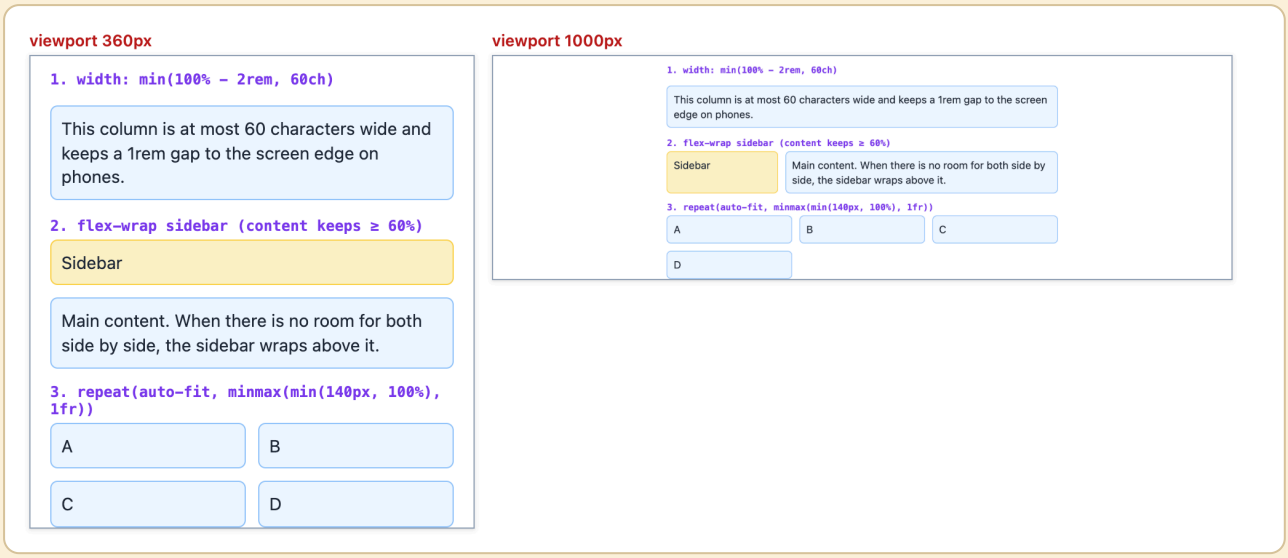


Figure 10.6 — The same page at 360px and 1000px.

Output of `code/07_intrinsic.mjs`:

```
 360px  wrapper 328.0px | sidebar wrapped above content | grid 2 column(s)
 480px  wrapper 448.0px | sidebar beside content (150.3px + 287.7px) | grid 3 column(s)
1000px  wrapper 529.1px | sidebar beside content (150.3px + 368.8px) | grid 3 column(s)
```

By hand at 480px: wrapper = min(480 − 32, 60ch ≈ 529px) = 448px. The line needs 150 + 10 + 0.6 × 448 = 428.8px ≤ 448, so the sidebar stays beside; free space 448 − 10 − 150 = 288px is shared 1 : 999, so the sidebar gets 150 + 0.288 = 150.3px ✓. At 360px the wrapper is 328px and the line would need 150 + 10 + 196.8 = 356.8px > 328, so it wraps ✓.

Other intrinsic tools

- `flex-wrap: wrap` + `flex: 1 1 12rem` on items — a row of buttons that becomes a column when needed (Chapter 8).
- `max-inline-size: 65ch` on text — the **measure** (line length) that reads best (Chapter 11).
- `aspect-ratio` for media placeholders; `object-fit: cover` to crop images into them (Chapter 11).

When not to use it

When the design genuinely needs a different structure (e.g. a sidebar that becomes a bottom tab bar), a media query states that intent more clearly than a clever flex trick.

10.10 Testing responsive design

What to test

Check	How	Pass criterion
No sideways scrolling	Resize from 320px to 1920px+	`scrollWidth ≤ clientWidth` at every width (except data tables, maps, etc.)
Reflow (WCAG 2.2 SC 1.4.10, AA)	View at 320 CSS px wide (= 1280px at 400% zoom)	All content and functions available with vertical scrolling only
Resize text (SC 1.4.4, AA)	Browser zoom 200%; also raise the default font size	No clipped or overlapping text, nothing lost
Text spacing (SC 1.4.12, AA)	Apply line-height 1.5, paragraph spacing 2em, letter 0.12em, word 0.16em	Nothing clipped — avoid fixed heights on text boxes
Target size (SC 2.5.8, AA, new in 2.2)	Inspect tap targets	At least 24 × 24 CSS px (or enough spacing)
Orientation (SC 1.3.4)	Rotate the device	Works in both, unless essential
Touch vs mouse	Real phone; `(hover: none)`	Nothing is reachable *only* on hover
Preferences	Emulate dark, reduced motion, forced colours	Readable, no lost UI

Tools

- **DevTools device mode**: Chrome/Edge (Ctrl/Cmd+Shift+M), Firefox Responsive Design Mode (Ctrl/Cmd+Shift+M), Safari Responsive Design Mode (Develop menu). Drag the width handle slowly — you'll spot the widths where things break. Chrome's Rendering panel emulates `prefers-color-scheme`, `prefers-reduced-motion`, `forced-colors` and more; Elements shows a `container` badge and lets you inspect container queries.
- **Real devices**: emulators don't reproduce touch precision, the dynamic toolbar (`dvh`), the on-screen keyboard or real CPU speed. Test at least one Android phone and one iPhone; remote debugging (Chrome `chrome://inspect`, Safari Web Inspector) attaches DevTools to them.
- **Automated screenshots and checks** in CI with Puppeteer or Playwright, like this chapter's scripts.

An automated overflow test

`code/08_overflow_check.mjs` opens pages at 320/768/1280px and reports horizontal overflow and the elements sticking out:

```
const check = page => page.evaluate(() => {
  const vw = document.documentElement.clientWidth;
  const culprits = [...document.body.querySelectorAll('*')]
    .filter(el => el.getBoundingClientRect().right > vw + 0.5 || /* … content overflowing its box … */)
    .map(el => el.tagName.toLowerCase() + (el.className ? '.' + el.className : ''));
  return { overflow: document.documentElement.scrollWidth > vw, scrollWidth: document.documentElement.scrollWidth, culprits };
});
```

```
02_mobile_first.html   320px  ok
02_mobile_first.html   768px  ok
02_mobile_first.html  1280px  ok
08_broken.html         320px  OVERFLOW (page is 575px wide): div.promo, img, p
08_broken.html         768px  ok
08_broken.html        1280px  ok
08_fixed.html          320px  ok
08_fixed.html          768px  ok
08_fixed.html         1280px  ok
```

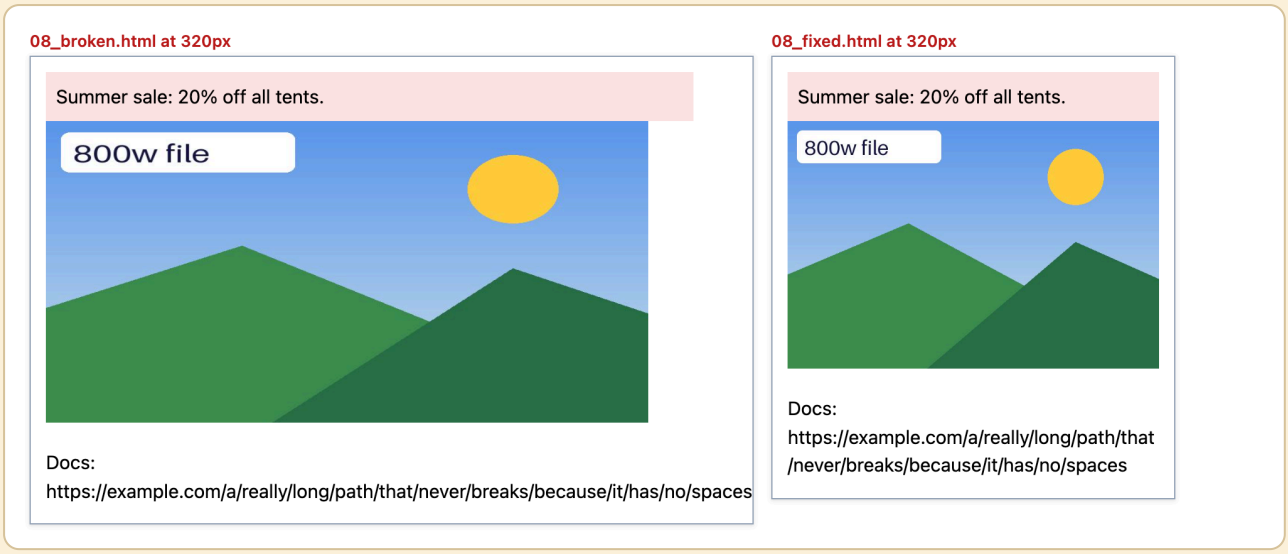


Figure 10.7 — The three most common causes of sideways scrolling on phones, and their one-line fixes.

Bug in `08_broken.html`	Fix in `08_fixed.html`
`.promo { width: 500px }`	`max-width: 500px` — fixed widths are the #1 cause
`<img width="480">` with no CSS	`img { max-width: 100%; height: auto; }`
Long URL with no break opportunities	`overflow-wrap: anywhere` (or `break-word`) on text containers

Other frequent culprits: `100vw` widths (10.2), grid columns with `1fr` that can't shrink below a long word (`minmax(0, 1fr)`, §9.10), flex items with `min-width: auto` (Chapter 8), `<pre>` blocks and wide tables (wrap them in `overflow-x: auto`).

In the interview

30-second answer: "I build mobile-first: the base CSS is the single-column phone layout and `min-width` media queries add columns where the content needs them. The viewport meta tag makes phones lay out at device width. I use intrinsic techniques — flex-wrap, `repeat(auto-fit, minmax())`, `clamp()` for fluid type with a `rem` term — so I need few breakpoints, and container queries for components that live in different-sized slots. I respect user preferences: `prefers-color-scheme` with `color-scheme`, `prefers-reduced-motion`. Images use `srcset` / `sizes`. I test in device mode, on real phones, at 320px and 200% zoom for WCAG reflow."

Follow-ups:

- *Why does `100vh` misbehave on mobile?* → It equals the large viewport (toolbars hidden), so with toolbars visible the bottom is cut off. Use `svh` or `dvh`.
- *How do you pick breakpoints?* → Where the content breaks, not device widths; prefer em so they follow the user's font size.

- *Media vs container queries?* → Viewport vs ancestor size; page layout vs component layout.
- *How do you avoid horizontal scrolling?* → No fixed widths (use `max-width`), flexible media, `overflow-wrap`, `minmax(0,1fr)`, and an automated check at 320px.

Self-check

1. What width does WCAG's Reflow criterion test? → 320 CSS px (equivalent to 1280px at 400% zoom).
2. Name three causes of horizontal overflow. → Fixed widths, images without `max-width`, long unbreakable strings (also `100vw`, rigid grid tracks, wide tables).
3. What can't DevTools device mode emulate well? → Real touch, dynamic toolbars/`dvh`, on-screen keyboard, real performance.

Chapter 10 TL;DR

- **Responsive** = fluid layout + flexible media + conditional CSS. Build **mobile-first**: base CSS is the phone layout; `(width >= …)` queries add columns.
- `<meta name="viewport" content="width=device-width, initial-scale=1">` — without it phones lay out at ~980px and shrink (measured scale 0.408 on a 400px phone). Never disable zoom.
- **Viewport units**: `svh` (toolbars shown), `lvh` (hidden), `dvh` (live); `vh` ≈ `lvh` on phones; `100vw` includes the scrollbar.
- **Media queries**: type + features + `and` / `,` / `not`. Range syntax `(640px <= width < 1024px)` is widely available and removes boundary gaps. `em` in queries = browser default font size. JS: `matchMedia(q)` + `change` event.
- **Preferences**: `prefers-color-scheme` + `color-scheme: light dark` (themes form controls too) + token overrides; `light-dark()` (Baseline 2024); `prefers-contrast`, `forced-colors`, `prefers-reduced-motion`.
- **Container queries** (widely available since Aug 2025): `container: name / inline-size` on a wrapper, `@container name (width >= 400px)` for descendants, `cqi` units. Style queries on custom properties: Baseline 2026. Scroll-state queries: Chromium only.
- **Fluid type**: `clamp(MIN, a·rem + b·vw, MAX)`; slope = Δsize/Δviewport, intercept = size − slope × viewport; keep a `rem` term so user settings and zoom still work (vw-only stayed 16px when the default was 20px).
- **Images**: `max-width: 100%`, `srcset`/`sizes`, `<picture>`; `image-set()` for backgrounds (downloads one file per DPR).
- **Intrinsic layouts**: `min(100% - 2rem, 60ch)`, flex-wrap sidebar, `repeat(auto-fit, minmax(min(X,100%),1fr))` — breakpoints chosen by content.
- **Test**: 320px reflow, 200% zoom, text spacing, 24px targets, real devices, dark/reduced motion, automated overflow checks.

Next: Chapter 11 — Typography, Colour and Visual Effects.